TASK 9: Main project - Using ansible

Description

Instructions:

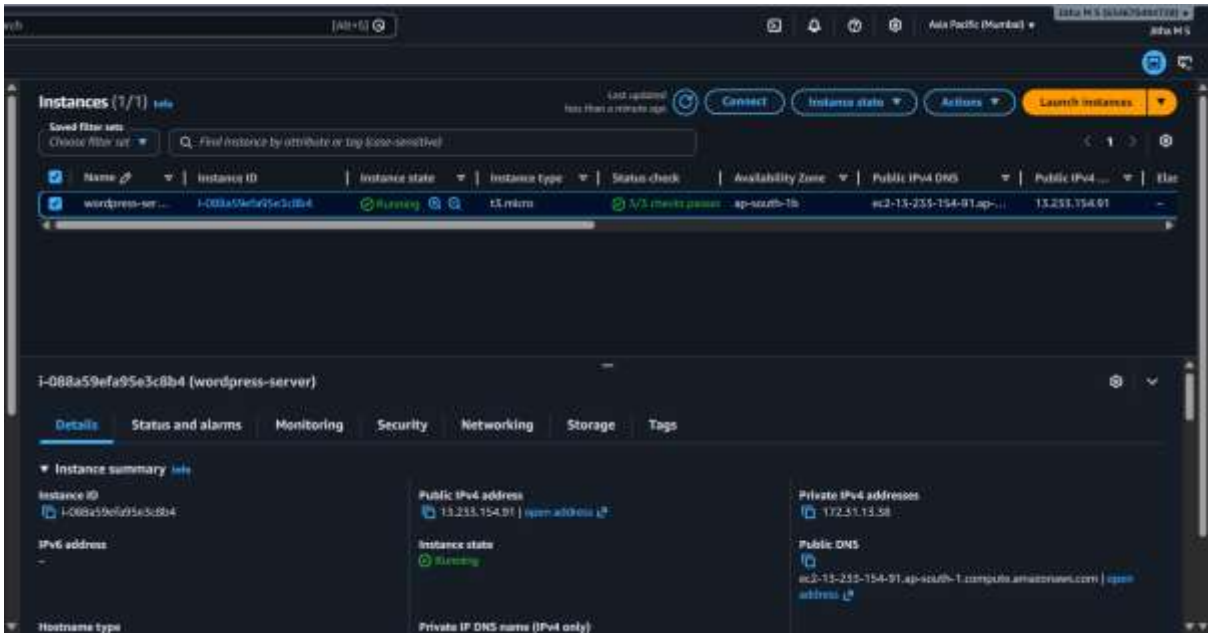
1. Install the necessary packages, which include MySQL 10.6, PHP version 8 or higher, and Nginx on your Linux server.
   2. Create a user's home directory under the '/home' directory. You should set the root directory for the website to be '/home/username/websitename/public.'
   3. Configure SFTP (Secure File Transfer Protocol) access for the user to ensure secure file management.
   4. Configure phpmyadmin access for the user to ensure secure database management.
   4. Install a free SSL certificate to enable secure HTTPS access to your website.
   5. Once the website is hosted, proceed to the WordPress dashboard to make necessary configurations and customizations.
   6. Write a personal blog post about yourself using the WordPress content management system.

Objective:

This project demonstrates the deployment of a WordPress website on an Amazon Linux 2023 server using AWS EC2. The project includes the installation and configuration of Nginx, PHP, MariaDB, phpMyAdmin, SFTP, and SSL services. The objective is to create a secure and fully functional WordPress hosting environment.

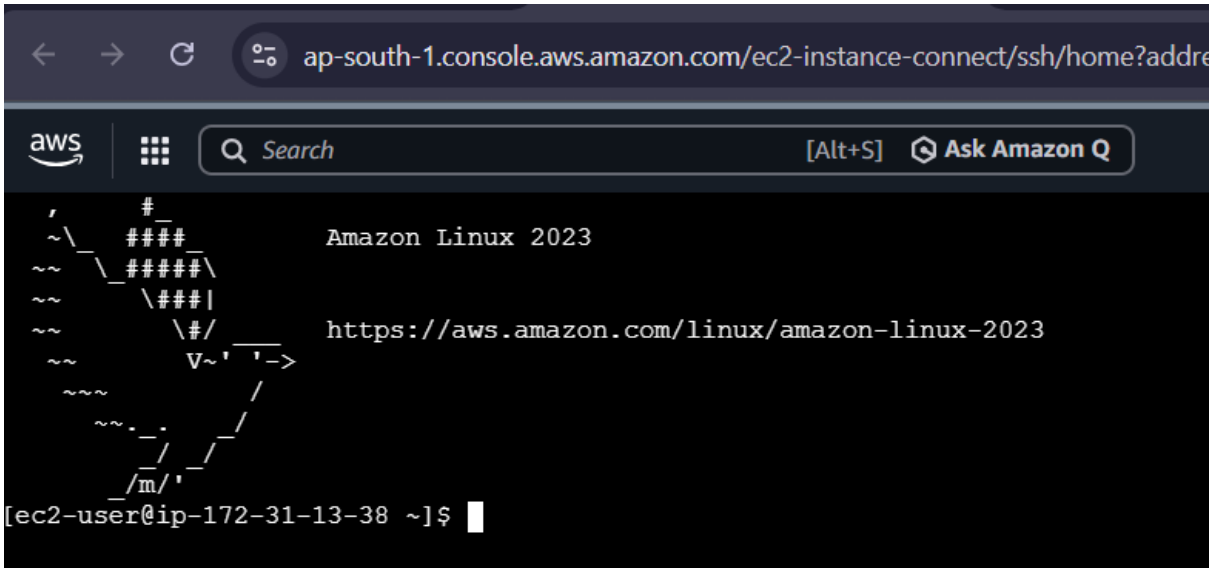
Step 1: EC2 Instance Running State

The EC2 instance was successfully launched and verified in the running state. The public IP address was used for server access and website hosting.



Step 2: SSH Connection to EC2

A secure SSH connection was established from the local system to the EC2 instance. This allowed remote administration and configuration of the server.



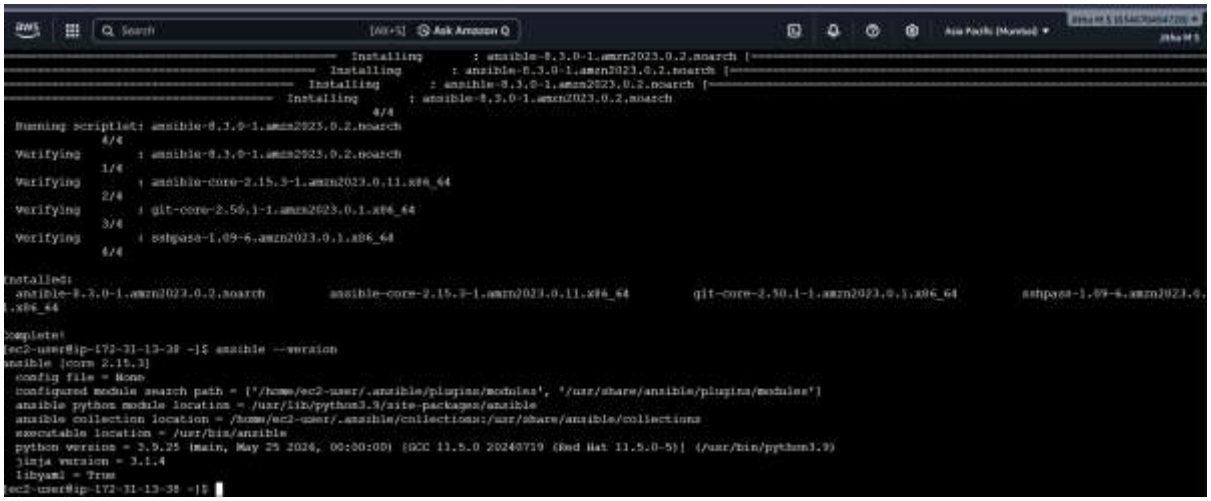
Step 3: System Update

The server packages were updated to the latest available versions using the DNF package manager. This ensured that the system was secure and ready for software installation.

```
[ec2-user@ip-172-31-13-38 ~]$ sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository                                   [===
         Amazon Linux 2023 Kernel Livepatch repository
                                                                                  3
99 kB/s |  47 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-13-38 ~]$
```

Step 4: Install Ansible

Ansible was installed on the Amazon Linux server to automate configuration and deployment tasks. The installation was verified by checking the installed Ansible version.



Step 5: Create Ansible Project Directory

A dedicated project directory was created to store Ansible configuration files, inventory files, and playbooks. This helped organize the automation project efficiently.

```
[ec2-user@ip-172-31-13-38 ~]$ mkdir wordpress-project
[ec2-user@ip-172-31-13-38 ~]$ cd wordpress-project
[ec2-user@ip-172-31-13-38 wordpress-project]$ pwd
/home/ec2-user/wordpress-project
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 6: Create Inventory File

An inventory file was created to define the target host for Ansible automation. The inventory acts as a list of servers that Ansible can manage.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ vim inventory
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat inventory
[web]
localhost ansible_connection=local
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

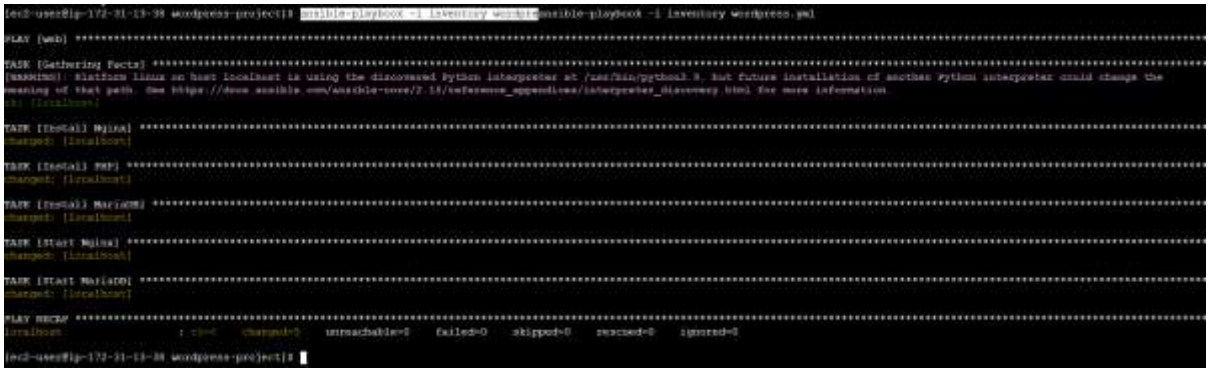
Step 7: Create Ansible Playbook

An Ansible playbook was created to automate the installation and configuration of required services. The playbook contains tasks that simplify server setup and deployment.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ vim wordpress.yml
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat wordpress.yml
---
- hosts: web
  become: yes

  tasks:

  - name: Install Nginx
    dnf:
      name: nginx
      state: present

  - name: Install PHP
    dnf:
      name:
        - php
        - php-fpm
        - php-mysqlnd
      state: present

  - name: Install MariaDB
    dnf:
      name:
        - mariadb105-server
      state: present

  - name: Start Nginx
    service:
      name: nginx
      state: started
      enabled: yes

  - name: Start MariaDB
    service:
      name: mariadb
      state: started
      enabled: yes
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 8: Execute Ansible Playbook

The Ansible playbook was executed against the target server. Successful execution ensured that all required packages and configurations were applied automatically.



Step 9: Create Website User

A dedicated user account named **alice** was created to manage website files securely. A password was configured for authentication and SFTP access.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo useradd alice
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo passwd alice
Changing password for user alice.
New password:
BAD PASSWORD: The password is shorter than 8 characters
Retype new password:
passwd: all authentication tokens updated successfully.
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

Step 10: Create Website Directory Structure

The website root directory was created under /home/alice/myblog/public as specified in the project requirements. This directory serves as the document root for the WordPress website.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo mkdir -p /home/alice/myblog/public
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo chown -R alice:alice /home/alice/myblog
[ec2-user@ip-172-31-13-38 wordpress-project]$ ls -l /home/alice
ls: cannot open directory '/home/alice': Permission denied
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo ls -l /home/alice
total 0
drwxr-xr-x. 3 alice alice 20 Jun 10 06:44 myblog
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo ls -l /home/alice/myblog/public
total 0
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

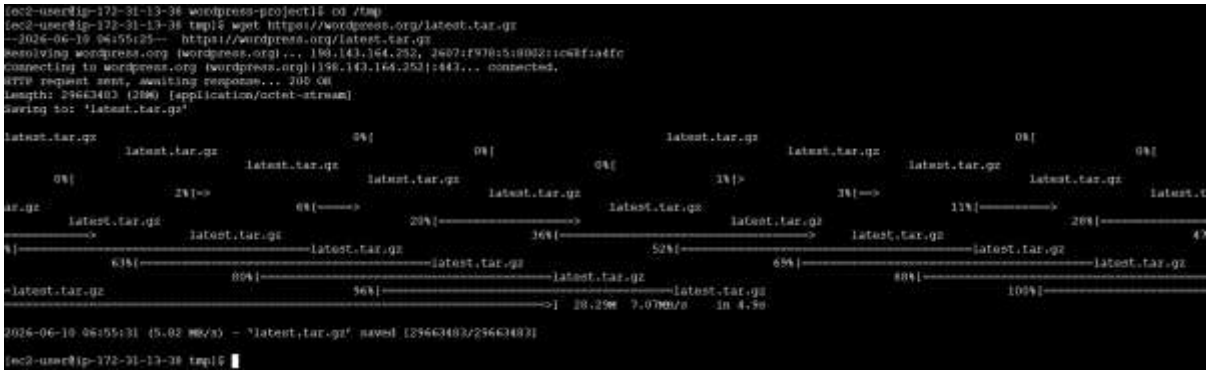
Step 11: Create Nginx Virtual Host

A custom Nginx virtual host configuration was created for hosting the WordPress website. The configuration specifies the domain name, document root, and PHP processing settings.

```
[ec2-user@ip-172-31-13-38 wordpress-project]$ sudo vim /etc/nginx/conf.d/myblog.conf
[ec2-user@ip-172-31-13-38 wordpress-project]$ cat sudo vim /etc/nginx/conf.d/myblog.conf
cat: sudo: No such file or directory
cat: vim: No such file or directory
server {

listen 80;

server_name yourdomain.com www.yourdomain.com;

root /home/alice/myblog/public;

index index.php index.html;

location / {
try_files $uri $uri/ /index.php?$args;
}

location ~ \.php$ {
fastcgi_pass unix:/run/php-fpm/www.sock;
include fastcgi_params;
fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
}
[ec2-user@ip-172-31-13-38 wordpress-project]$
```

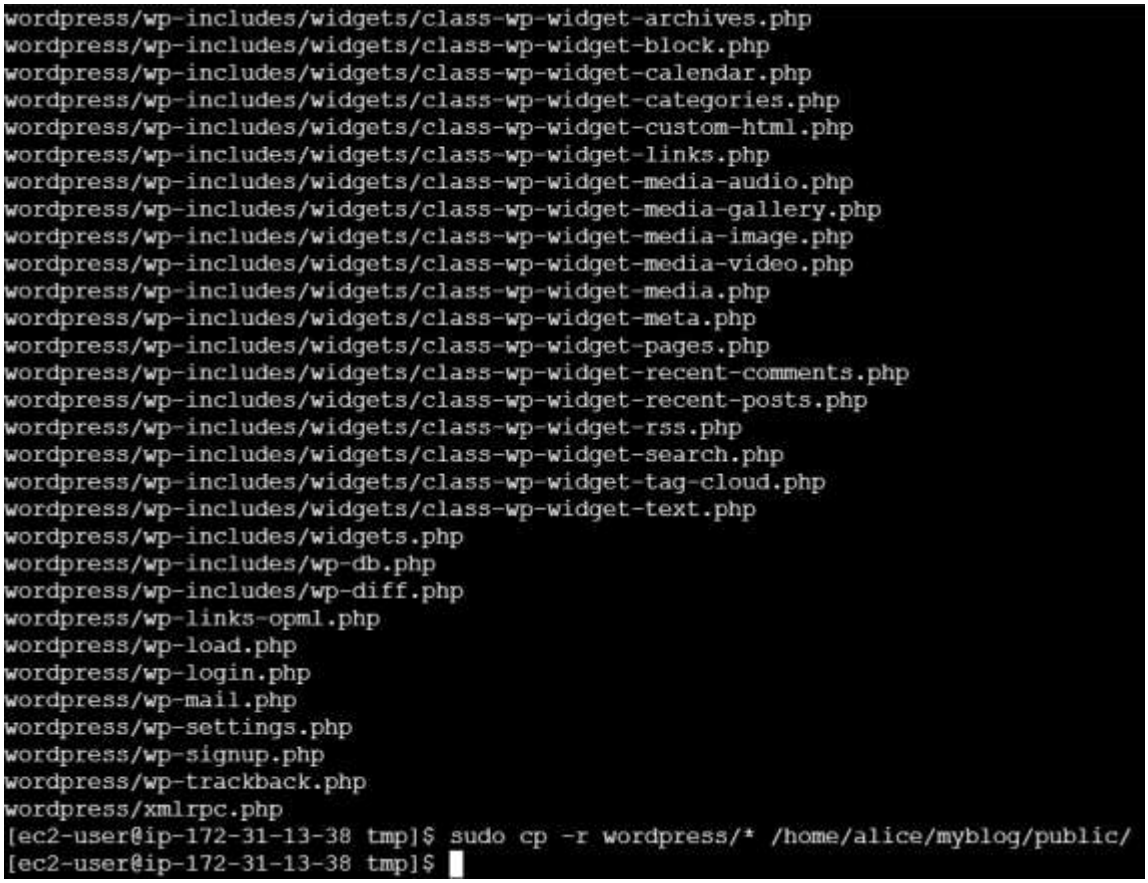
Step 12: Download WordPress

The latest version of WordPress was downloaded from the official WordPress website. The package contains all files required to deploy the content management system.



Step 13: Extract and Copy WordPress Files to Website Root

The downloaded WordPress archive was extracted and copied to the website root directory. Appropriate permissions were configured to allow access by the web server.



Step 14: Create WordPress Database

A MariaDB database was created to store WordPress content and configuration data. A dedicated database user was also configured with the required privileges.

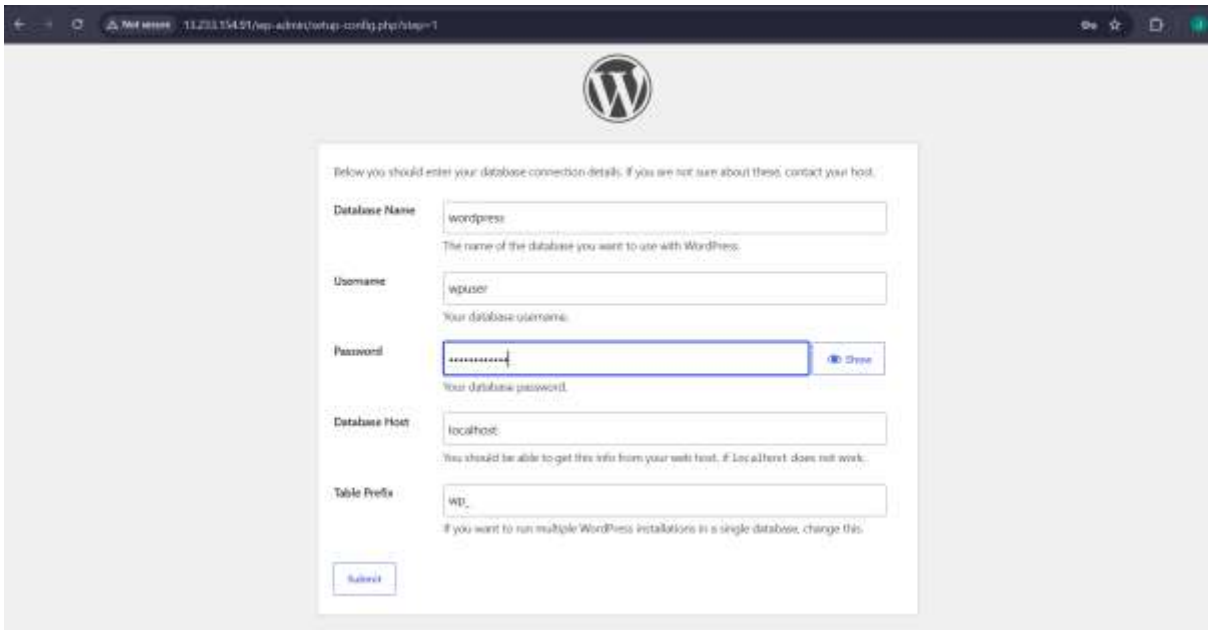
```
[ec2-user@ip-172-31-13-38 tmp]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE wordpress;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> CREATE USER 'wpuser'@'localhost'
    -> IDENTIFIED BY 'Password@123';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]>
MariaDB [(none)]> GRANT ALL PRIVILEGES
    -> ON wordpress.*
    -> TO 'wpuser'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [(none)]>
MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.000 sec)

MariaDB [(none)]>
MariaDB [(none)]> EXIT;
```

Step 15: Configure wp-config.php

The WordPress configuration file (wp-config.php) was updated with the database connection details. This allows WordPress to communicate with the MariaDB database server.

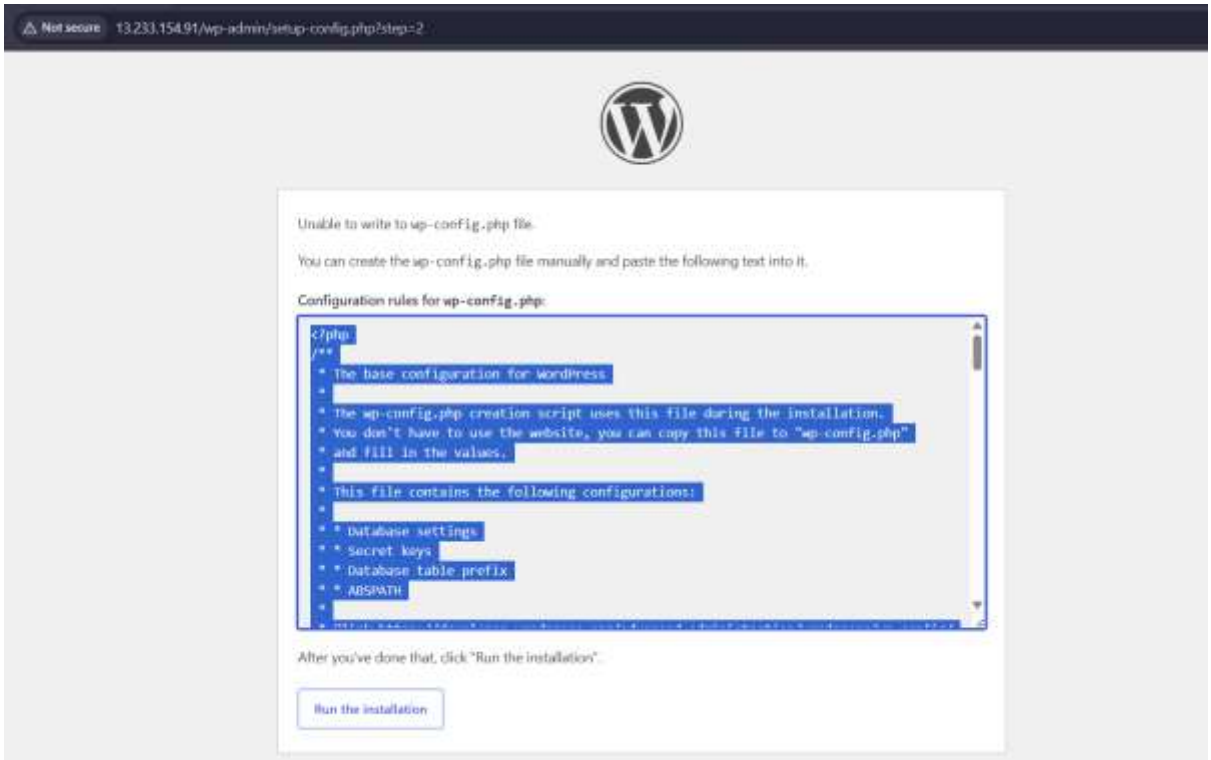
Step 16: WordPress Database Configuration

Database credentials including database name, username, password, and host information were entered during the WordPress setup process. This completed the database connectivity configuration.



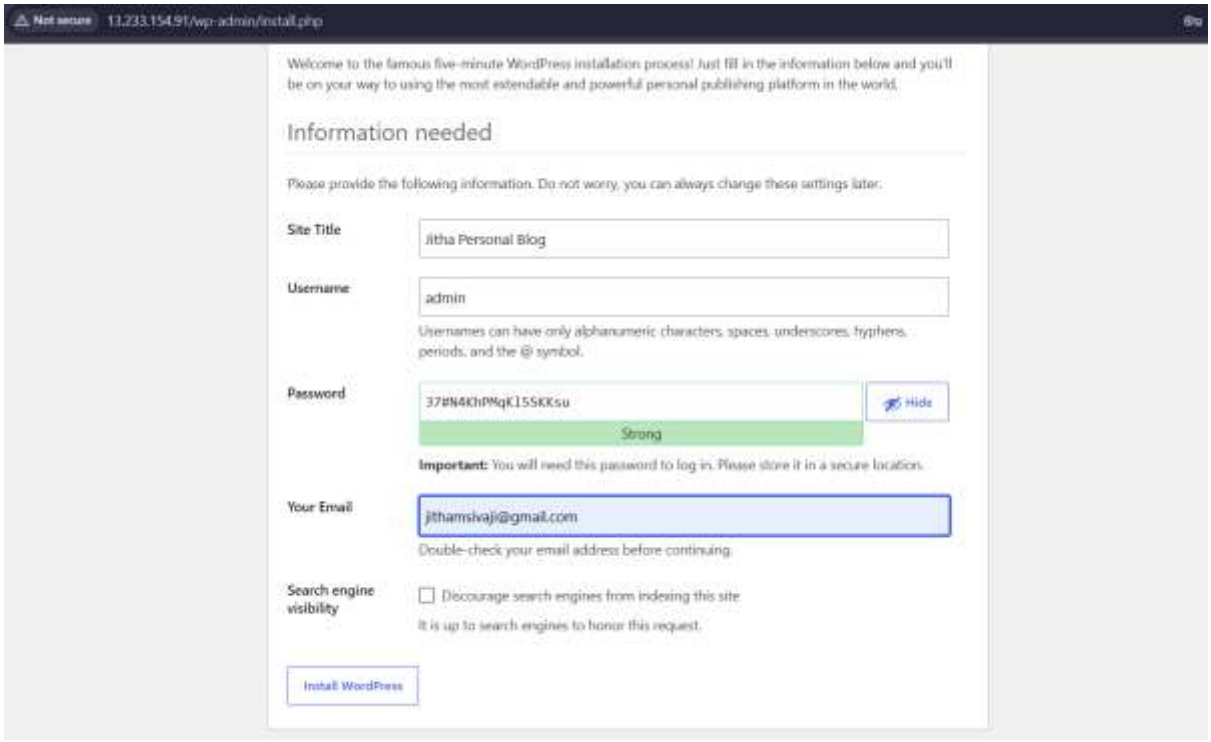
Step 17: WordPress Configuration File

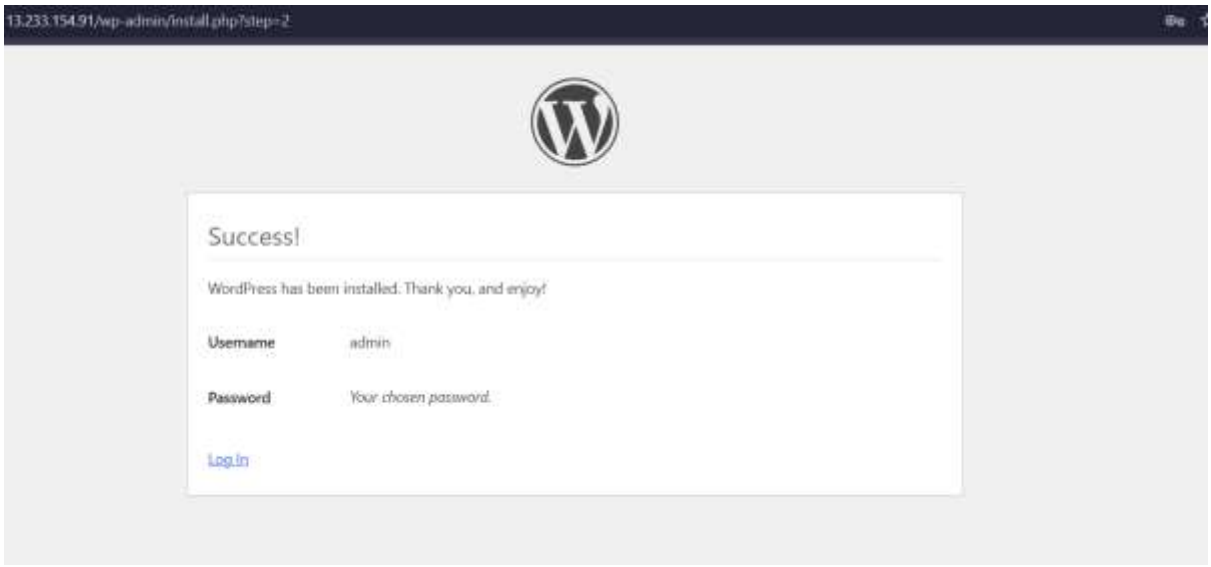
The WordPress configuration file was generated and verified successfully. This file contains the database settings and essential application configuration parameters.



Step 18: WordPress Installation Page

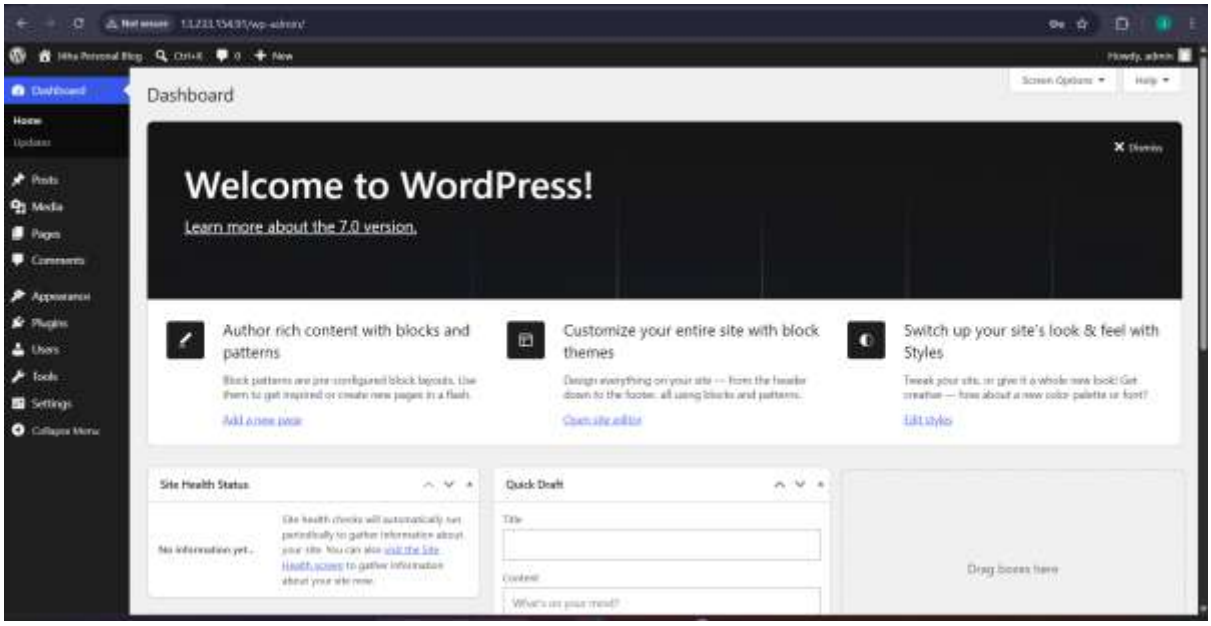
The WordPress installation wizard was successfully accessed through a web browser. Site information and administrator account details were configured during this step.





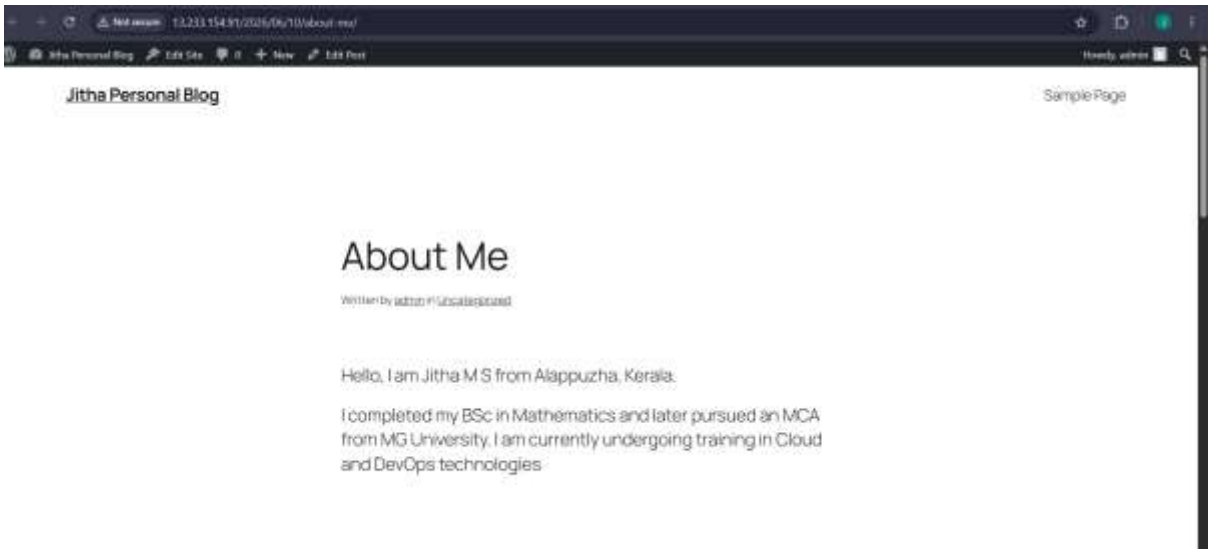
Step 19: WordPress Dashboard

After successful installation, the WordPress administrative dashboard was accessed. The dashboard provides complete control over website content, appearance, and settings.



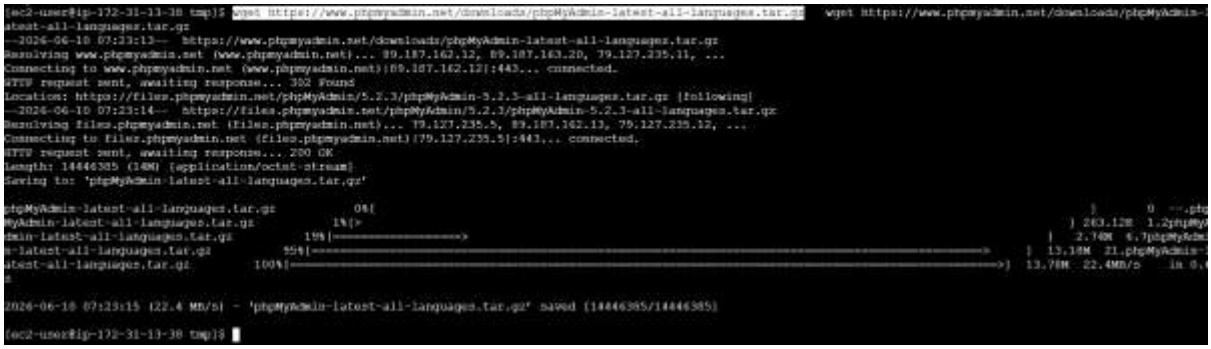
Step 20: Create Blog Post

A personal blog post was created and published using the WordPress content management system. This verified that the website was functioning correctly and ready for content publishing.



Step 21: Download phpMyAdmin

The latest phpMyAdmin package was downloaded from the official website. This tool provides a graphical interface for managing MySQL and MariaDB databases.



Step 22: Extract phpMyAdmin

The downloaded phpMyAdmin archive was extracted on the server. The extracted files were prepared for deployment within the web server environment.

Step 23: Configure phpMyAdmin

phpMyAdmin was configured and integrated with the Nginx web server. Appropriate settings were applied to allow secure database management through a browser.

Step 24: Verify Nginx Configuration

The Nginx configuration was tested to ensure there were no syntax or configuration errors. Successful validation confirmed that the web server was correctly configured.

```
[ec2-user@ip-172-31-13-38 tmp]$ sudo chown -R nginx:nginx /usr/share/nginx/html/phpmyadmin
[ec2-user@ip-172-31-13-38 tmp]$ sudo vim /etc/nginx/conf.d/phpmyadmin.conf
[ec2-user@ip-172-31-13-38 tmp]$ cat sudo vim /etc/nginx/conf.d/phpmyadmin.conf
cat: sudo: No such file or directory
cat: vim: No such file or directory
location /phpmyadmin {
    root /usr/share/nginx/html;
    index index.php;
}

location ~ ^/phpmyadmin/(.+\.php)$ {
    root /usr/share/nginx/html;
    fastcgi_pass unix:/run/php-fpm/www.sock;
    include fastcgi_params;
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
}
[ec2-user@ip-172-31-13-38 tmp]$
```

Step 25: Access phpMyAdmin

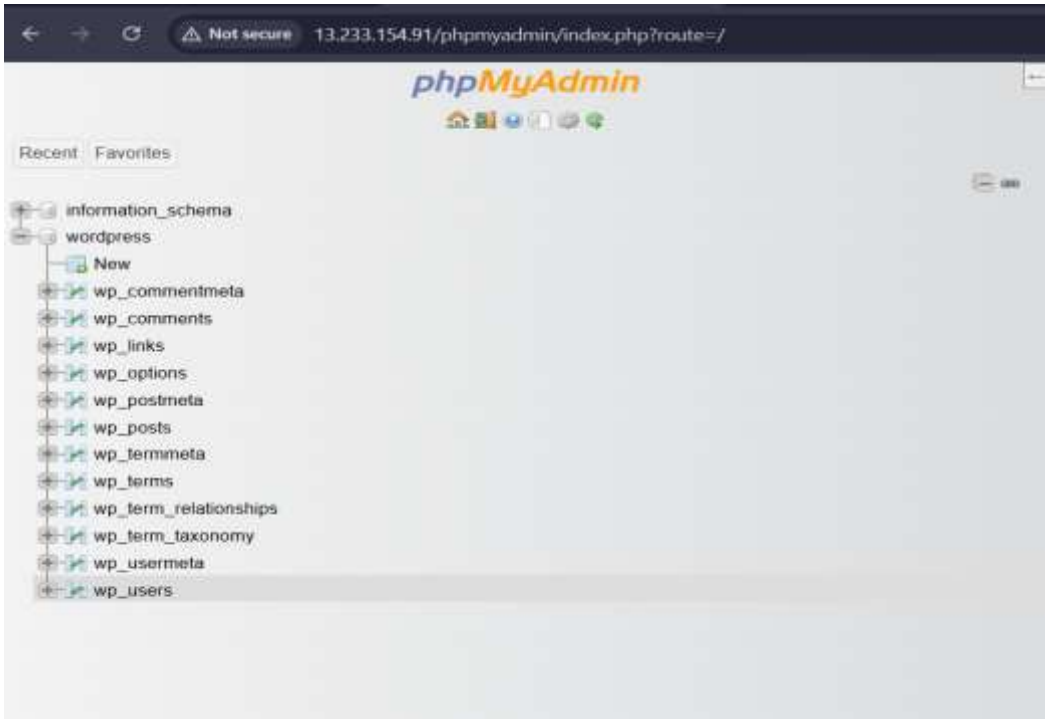
The phpMyAdmin web interface was successfully accessed through the browser. This confirmed that the installation and configuration were completed successfully.

Step 26: Login to phpMyAdmin

Authentication was performed using the configured database user credentials. Successful login verified database connectivity and user permissions.

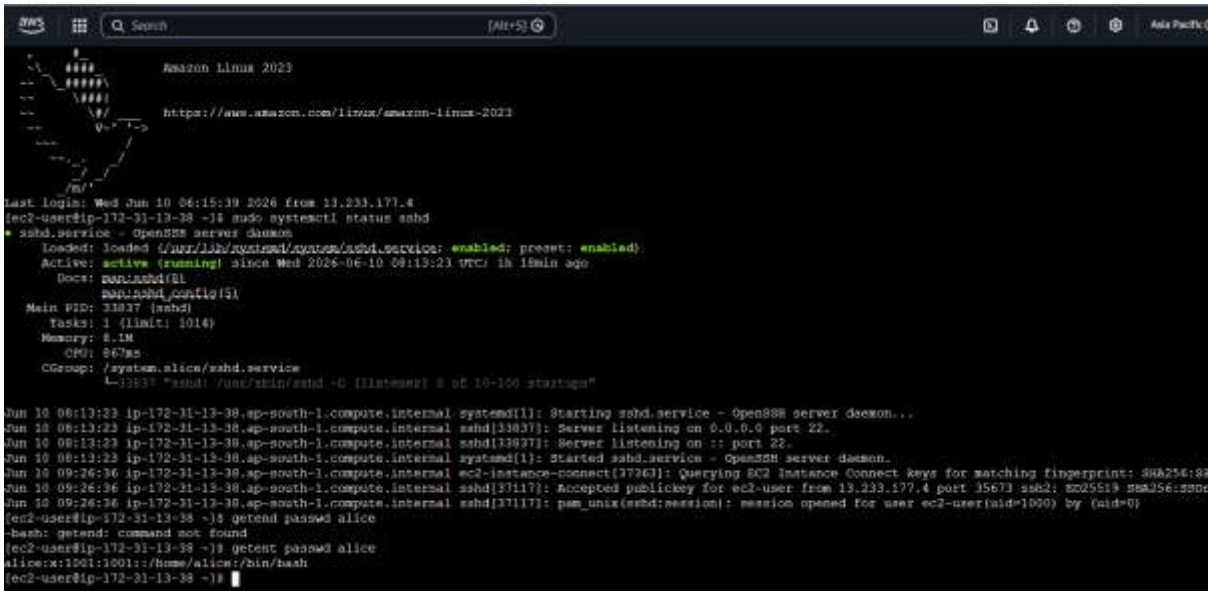
Step 27: phpMyAdmin Dashboard

The phpMyAdmin dashboard was accessed successfully. The available databases and management options were verified through the graphical interface.



Step 28: SFTP Configuration

SFTP access was configured for the user **alice** to enable secure file transfer and website management. This ensures secure communication between the client and server.



Step 29: Domain DNS Configuration

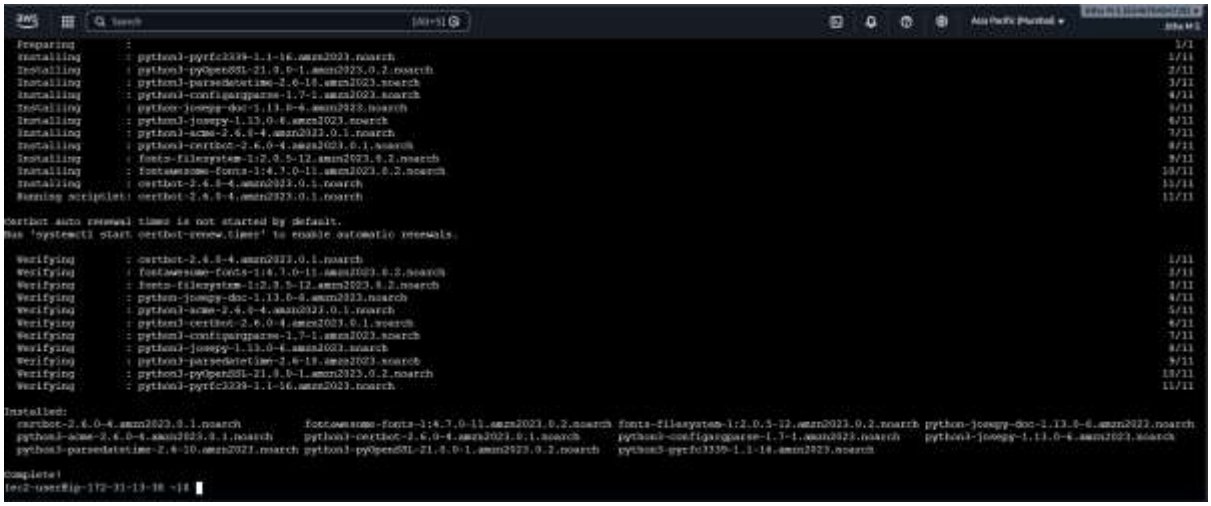
A DuckDNS domain was created and configured to point to the EC2 instance public IP address. DNS resolution was verified to ensure proper domain connectivity.

Step 30: WordPress Website Access Verification Using Domain Name

The WordPress website was successfully accessed using the configured domain name. This confirmed that DNS records and web server settings were functioning correctly.

Step 31: Install Certbot

Certbot was installed to obtain and manage free SSL certificates from Let's Encrypt. This tool simplifies SSL certificate generation and renewal.



Step 32: HTTPS Verification

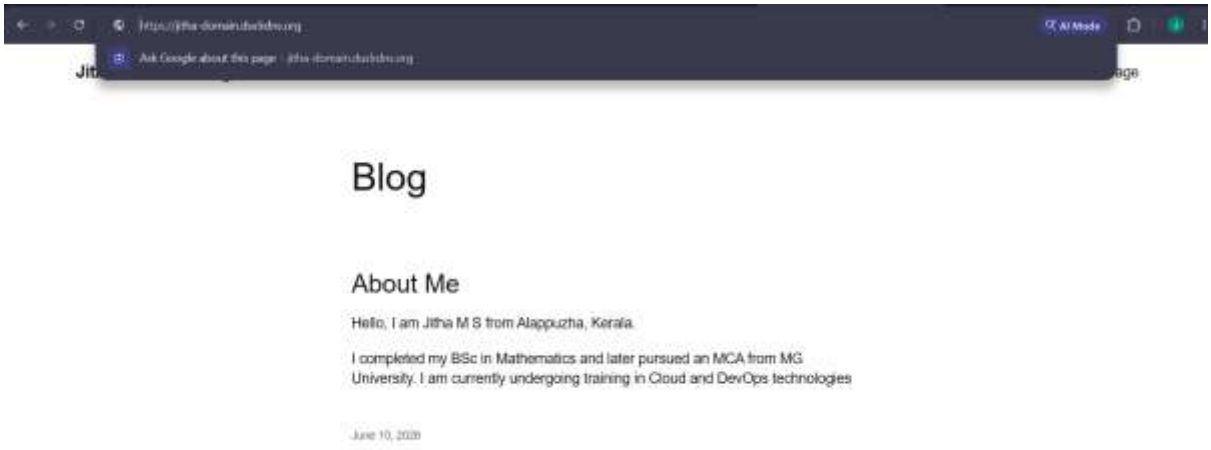
The website was tested using HTTPS to verify secure communication. Browser verification confirmed that SSL encryption was functioning correctly.

Step 33: Generate SSL Certificate

A free SSL certificate was successfully generated and installed using Certbot. The certificate was linked to the configured domain name and web server.

Step 34: HTTPS Access Verification After SSL Installation

The website was successfully accessed using HTTPS after SSL configuration. The secure connection verified that encrypted communication was enabled for all users.



Conclusion:

This project successfully demonstrates the deployment of a WordPress website on Amazon Linux 2023 using AWS EC2 and Ansible. Nginx, PHP, MariaDB, phpMyAdmin, SFTP, domain configuration, and SSL security were implemented successfully. The website was verified through both HTTP and HTTPS access, and a personal blog post was published to confirm complete functionality.